



Universidade Federal do Cariri
Ciência da Computação
Juazeiro do Norte, CE
2025

Computação Gráfica

Material de Apoio V: Introdução à OpenGL com C

Monitor(a): Felipe Coêlho
Profª Responsável: Luana Batista



Sumário

1.	Preparação do ambiente	3
2.	Introdução ao OpenGL	8
3.	Primeiro exemplo: Nosso Hello World com OpenGL	19

1 - Preparação do Ambiente

1.1 Instalando o OpenGL no Windows

- Faça o download do MinGW em <https://www.mingw-w64.org>.
- Ou, para usuários do Windows 10, use o link alternativo: https://drive.google.com/drive/folders/1-rRkvCay8gRyIh5ZZNpYVvOQPz1Z_82q.
- O MinGW fornece um ambiente de desenvolvimento no Windows com suporte ao compilador GCC .
- Durante a instalação, escolha o diretório C:\MinGW como destino (referido como <MINGW_HOME>).
- Selecione e instale os seguintes pacotes essenciais:
 - **mingw32-base**
 - **mingw32-gcc-g++**
 - **msys-base**
- Após a instalação, adicione os diretórios abaixo à variável de ambiente Path do Windows:
 - **<MINGW_HOME>\bin**
 - **<MINGW_HOME>\msys\1.0\bin**

- O OpenGL geralmente já vem incluído nos sistemas operacionais Windows e Linux. No entanto, para desenvolver aplicações com OpenGL de forma prática, é recomendável utilizar bibliotecas complementares que auxiliem na criação de janelas e no gerenciamento da entrada do usuário. Essas bibliotecas oferecem funcionalidades extras, como suporte a janelas em tela cheia, redimensionamento e controle de eventos de teclado e mouse, entre outras.
- Para utilizá-las, é necessário fazer o download dos arquivos de cabeçalho (headers), das bibliotecas estáticas (libs) e das bibliotecas dinâmicas (bin), compatíveis com o seu sistema operacional.
- Para adicionar as bibliotecas GLU e FreeGLUT (GLUT) ao ambiente MinGW acesse o link e baixe os arquivos necessários:
<https://drive.google.com/drive/folders/1cQi8lQwe98hFjQcXezO6Hn2YaiFCaGWx>.
- Extraia o conteúdo do arquivo compactado que foi baixado.
- Mova os arquivos localizados na pasta: "include\GL" para o diretório: "<MINGW_HOME>\include\GL".
- Copie os arquivos da pasta: "lib" para: "<MINGW_HOME>\lib".
- Transfira os arquivos da pasta: "bin" para: "<MINGW_HOME>\bin".
- Caso prefira usar uma IDE para compilar seus programas em C, baixe e instale o Code::Blocks (na versão sem MinGW incluído) pelo site oficial: <https://www.codeblocks.org/>.

1.2 Instalando o OpenGL no Linux

Para configurar o ambiente de desenvolvimento OpenGL em sistemas Linux, é necessário instalar alguns pacotes (os nomes podem variar ligeiramente dependendo da distribuição utilizada):

- **freeglut**
- **freeglut-devel**
- **make**
- **gcc**

Além disso, se preferir utilizar uma IDE para desenvolver e compilar seus códigos em C, é recomendável instalar o Code::Blocks.

1.3 Vamos aos possíveis Erros Comuns

a) Ausência de algumas bibliotecas DLL do OpenGL. Para verificar, siga os passos abaixo:

- **No Windows:**
- Execute os seguintes comandos no terminal (Prompt de Comando):
 - **dir C:\Windows\System32\opengl32.dll**
 - **dir C:\Windows\System32\glu32.dll**
- Se os arquivos opengl32.dll e glu32.dll forem listados, isso confirma que o OpenGL já está presente no sistema.

- **No Linux:** Use o comando: **ls /usr/lib/libGL***

- A presença de arquivos listados indica que as bibliotecas do OpenGL estão disponíveis.

b) Confirme se o compilador MinGW está instalado corretamente. Para verificar, siga os passos abaixo:

- Para checar se o MinGW está instalado, execute no terminal o comando: **gcc --version** .
- Se o comando retornar a versão do GCC, isso indica que o compilador está instalado corretamente.
- Verifique o local de instalação do MinGW. Por padrão, ele costuma ser instalado em diretórios como: **C:\MinGW** ou **C:\MinGW-w64**
- O caminho pode variar de acordo com a versão utilizada.



c) Confirme a configuração do MinGW no CodeBlocks. Acesse o menu “Settings” e selecione a opção “Compiler”.

- Na janela que se abrir, vá até a aba “Toolchain executables” e confira se o caminho apontado corresponde ao local onde o compilador MinGW foi instalado. Se o caminho estiver incorreto, atualize com o diretório correto da instalação.

1.4 Diretivas de compilação

As diretivas de compilação são instruções fornecidas ao compilador durante a compilação do código, indicando quais bibliotecas devem ser vinculadas ao programa.

Embora o OpenGL seja uma biblioteca multiplataforma, as diretivas variam conforme o sistema operacional.

Para compilar projetos com OpenGL corretamente, é necessário adicionar as diretivas apropriadas de acordo com o sistema utilizado. Coloque exatamente desse jeito pois é aqui que geralmente ou esqueçam de colocar e não funciona ou pode dar errado se não colocarem igual.

- **No Windows:**

“-lopengl32 -lglu32 -lfreetglut”

- **No Linux:**

“-lGL -lglut -lGLU”



2 - Introdução ao OpenGL

2.1 Visão Geral

OpenGL é uma Interface de Programação de Aplicativos (API), ou seja, um conjunto de funções que os desenvolvedores podem utilizar diretamente em seus programas.

Um dos principais benefícios é a portabilidade: o mesmo código em OpenGL pode ser executado em diferentes sistemas operacionais.

A API define um modelo funcional de como essas rotinas trabalham juntas para produzir imagens gráficas.

O programador interage apenas com essa interface de alto nível, sem precisar se preocupar com detalhes de baixo nível do hardware.

OpenGL abstrai as especificidades da placa gráfica (como NVIDIA ou AMD) e também do sistema operacional.

A biblioteca é capaz de gerar tanto gráficos 3D complexos quanto visualizações 2D mais simples, facilitando o desenvolvimento gráfico em múltiplas plataformas.

2.2 Ambiente de Execução para Programas Gráficos

OpenGL fornece um ambiente composto por um monitor (dispositivo de exibição) e uma biblioteca de software usada para desenhar formas gráficas na tela.

A API funciona como uma espécie de “caixa preta”:

- Entradas: comandos (funções) enviados pelo código do usuário, dados vindos de dispositivos de entrada (como mouse e teclado), entre outros.
- Saídas: a imagem gerada e apresentada na tela.

A interface é descrita com base nas funções que oferece: o programador faz chamadas em C utilizando a biblioteca OpenGL, e como resultado obtém-se uma janela gráfica que exibe o conteúdo solicitado com base nas entradas fornecidas.

2.3 API OpenGL

Normalmente, os programas desenvolvidos com OpenGL operam dentro de um gerenciador de janelas (window system).

As configurações iniciais envolvem:

- Definição do modo de exibição (display mode).
- Criação da janela de renderização (recorte).
- Estabelecimento de um sistema de coordenadas de referência, vinculado à janela.



A API disponibiliza uma grande quantidade de funções, cada uma com um propósito específico:

- Funções primitivas: determinam o que será desenhado (pontos, linhas, polígonos, etc);
- Funções de atributos: definem como o desenho será feito (cor, preenchimento, espessura, etc);
- E muitas outras, abrangendo desde configurações até interações e até mesmo animações.

2.4 Programação orientada a eventos

O OpenGL pode operar de forma orientada a eventos (event-driven), ou seja, o programa reage às ações do usuário ou do sistema, como:

- Cliques do mouse
- Pressionamento de teclas
- Redimensionamento da janela e muito mais

Futuramente nessa cadeira vocês terão que fazer projetos então poderão se deparar numa situação onde em um jogo, um clique do mouse pode selecionar um objeto, um pressionamento de tecla pode mover o personagem, e o movimento do mouse pode rotacionar a câmera. O OpenGL, em conjunto com um sistema de tratamento de eventos, permite que a aplicação reaja a esses eventos e atualize a cena de acordo.

2.5 Comandos do OpenGL

O OpenGL mantém o controle de diversas variáveis de estado, como a cor de fundo da janela, a cor usada para desenhar, o tamanho dos pontos, entre outras. Esses estados permanecem ativos até serem explicitamente modificados. Veja agora um exemplo de alguns comandos e suas funções:

- `glClearColor(r, g, b, a);` // Define cor de fundo, “a” vem de “alpha”
- `glColor3f(r, g, b);` // Cor usada para desenhar
- `glPointSize(3.0);` // Tamanho do ponto
- `glClear(GL_COLOR_BUFFER_BIT);` /* Limpa a janela alterando os bits do color buffer para refletir a cor de fundo. */

2.6 Estrutura de um Programa com OpenGL

Primeiramente vocês devem saber que o OpenGL geralmente é utilizado em conjunto com bibliotecas auxiliares para facilitar o desenvolvimento:

- **GLU (OpenGL Utility Library):** Fornece funcionalidades extras ao OpenGL, como: projeções, transformações de matrizes, cálculo de normais, mapeamento de texturas, entre outros recursos. Foi criada como uma extensão padrão do OpenGL, com o objetivo de tornar a interface mais amigável para os programadores. As funções GLU normalmente começam com o prefixo "glu" e são baseadas nas funções principais do OpenGL, garantindo compatibilidade com qualquer implementação OpenGL. É importante estabelecer um contexto OpenGL ativo antes de chamar funções GLU para evitar possíveis erros.

- **FreeGLUT (Free OpenGL Utility Toolkit):** Evolução do GLUT original, responsável por gerenciar janelas e facilitar a criação de superfícies de desenho.◦ Além de manter todas as funções do GLUT tradicional, o FreeGLUT oferece recursos adicionais como suporte a múltiplas janelas, modo de tela cheia, entrada 3D por teclado e mouse, e criação de contextos OpenGL com perfis avançados.

Vamos agora para um exemplo (**Figura 1**):

```
1  #include <windows.h>
2  #include <GL/glut.h>
3
4  int init(void) {
5      /* coloca as funções de inicialização */
6      /* inserir código */
7  }
8
9  void display(void) {
10     /* coloca as funções do OpenGL referentes a desenho */
11     /* inserir código */
12 }
13
14 int main(int argc, char** argv) {
15     /* inserir código */
16 }
```

Figura 1 - Código Exemplo.

- **#include <GL/glut.h> :** A biblioteca glut.h reúne os protótipos das funções utilizadas em programas com OpenGL, além de incluir automaticamente os arquivos de cabeçalho gl.h (OpenGL) e glu.h (GLU).

- **#include <windows.h>** : Para que o compilador reconheça que seu código é um programa utilizando OpenGL, é necessário indicar isso incluindo os cabeçalhos corretos. Logo, se estiver desenvolvendo no Windows, também é importante adicionar o cabeçalho windows.h antes de glut.h, pois ele fornece definições fundamentais para a plataforma.

Além das bibliotecas é necessário o código possuir três funções essenciais:

1. **init():** A função init (**Figura 2**) é responsável por configurar os parâmetros iniciais de renderização.

```
1 int init(void) {  
2     glClearColor(1.0, 1.0, 1.0, 0.0); //define a cor de fundo  
3  
4     glMatrixMode (GL_PROJECTION); //carrega a matriz de projeção  
5     gluOrtho2D(0.0,200.0,0.0,150.0); //define projeção ortogonal 2D  
6 }
```

Figura 2 - Código Exemplo.

- **glMatrixMode:** Função que determina qual matriz será modificada.
- **gluOrtho2D:** Função que define os limites da janela de recorte. Como a área de desenho funciona como um plano cartesiano infinito, é necessário especificar qual porção desse plano será exibida na tela do OpenGL, definindo os valores mínimos/máximos para x e y.

A função init é onde você configura as propriedades iniciais do ambiente de desenho do OpenGL. Ela define a cor de fundo onde você escolhe a cor que preencherá o plano de fundo da sua janela e também pode configurar a projeção onde se prepara o sistema de coordenadas 2D, definindo os limites (esquerda, direita, inferior, superior) da área visível onde seus objetos serão desenhados. Isso garante que o que você desenha apareça corretamente na tela.

2. display(): A função display (**Figura 3**) é usada como função de retorno (callback) para realizar o desenho na tela.

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT); //desenha o fundo (limpa a janela)  
  
    glColor3f(1.0,0.0,0.0); //altera o atributo de cor  
    glBegin(GL_LINES); //desenha uma linha  
        glVertex2i(180,15);  
        glVertex2i(10,145);  
    glEnd();  
  
    glFlush(); //desenha os comandos não executados  
}
```

Figura 3 - Código Exemplo.

Esta função é a "tela" do seu programa OpenGL. No código ocorre:

- **Limpa a janela:** Apaga o que estava desenhado antes, preparando para um novo desenho.
- **Define a cor:** Escolhe a cor que será usada para os próximos objetos.
- **Desenha uma linha:** Cria uma linha entre dois pontos específicos na tela.
- **Mostra o resultado:** Garante que todos os comandos de desenho sejam exibidos na tela.

3. main(): A função main (**Figura 4**) é a função principal que inicia a execução do programa.

```
1 int main(int argc, char** argv) {  
2     glutInit(&argc,argv); //inicializa o GLUT  
3     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //configura o modo de display  
4     glutInitWindowSize(400,300); //configura a largura e altura da janela de exibição  
5     glutInitWindowPosition(200,0);  
6     glutCreateWindow("Um programa OpenGL Exemplo"); //cria a janela de exibição  
7  
8     init(); //executa função de inicialização  
9     glutDisplayFunc(display); //estabelece a função "display" como a função callback de exibição  
10    glutMainLoop(); //mostre tudo e espere  
11    return 0;  
12 }
```

Figura 4 - Código Exemplo.

Esta é a função principal que "liga"/"gerencia" seu programa OpenGL:

- **Inicializa o OpenGL (GLUT):** Prepara a biblioteca para uso.
- **Configura a janela:** Define como a janela de exibição deve se comportar (tamanho, posição, modo de exibição).
- **Cria a janela:** Faz a janela aparecer na sua tela.
- **Chama init():** Executa as configurações iniciais que você definiu (como a projeção 2D).
- **Associa display():** Diz ao OpenGL para chamar sua função display() sempre que a janela precisar ser redesenhada.
- **Entra no loop principal:** Mantém a janela aberta e responsiva, esperando por eventos (como redimensionamento ou fechamento), e redesenhando conforme necessário.

O pipeline da OpenGL opera sempre em 3D, no entanto, é possível gerar gráficos em 2D configurando (**Figura 5**):

- **glMatrixMode(GL_PROJECTION) :** define o modo de projeção.
- **gluOrtho2D(xw_min, xw_max, yw_min, yw_max) :** delimita a área do plano cartesiano que será projetada na tela.

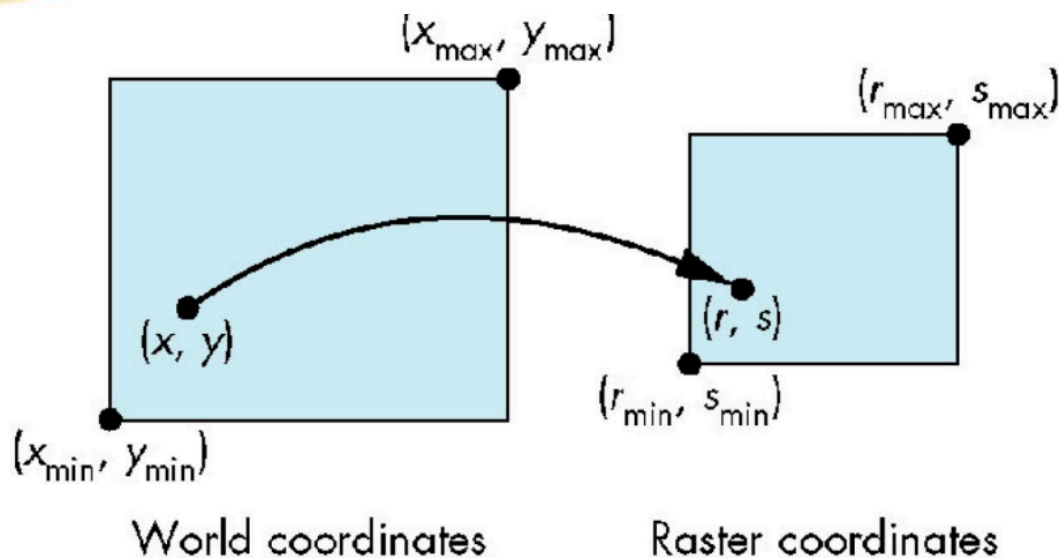


Figura 5 - Exemplo de projeção.

O OpenGL possibilita a criação de gráficos de forma independente do hardware utilizado. O programador pode definir os objetos em um Sistema de Coordenadas do Mundo, utilizando valores em ponto flutuante. A renderização, no entanto, ocorre no Sistema de Coordenadas do Dispositivo (ou da Tela), baseado em valores inteiros. A conversão entre esses dois sistemas, conhecida como projeção 2D ou visualização, é realizada automaticamente pelo OpenGL.

Por fim, vamos falar das **Primitivas Gráficas**. As primitivas fundamentais incluem pontos, linhas, polilinhas e polígonos (como `GL_POINTS`, `GL_LINES`, `GL_POLYGON`, entre outras).

Polígonos são compostos por linhas fechadas, constituídas exclusivamente por segmentos retos que só se encontram nas extremidades, podendo ser exibidos com ou sem preenchimento. Já as polilinhas são sequências de segmentos retos, com variados comprimentos e direções, e podem ser abertas ou fechadas. Para representar um objeto, o programador fornece uma sequência de vértices (**Figura 6**):


```

1 glBegin (GL_POINTS); //desenha 3 pontos
2     glVertex2i(100, 50);
3     glVertex2i(100, 130);
4     glVertex2i(150, 130);
5 glEnd();

```

Figura 6 - Código Exemplo.

- glVertex3f(x, y, z): Esta função é usada para especificar um vértice em 3 dimensões (x, y, z), onde os valores de cada coordenada são do tipo float. O 3 indica 3 dimensões e o f indica que os parâmetros são float.
- glVertex3fv(p): Esta é uma variação que também especifica um vértice em 3 dimensões, mas em vez de passar as coordenadas individualmente, ela recebe um ponteiro (p) para um vetor que contém as coordenadas (x, y, z). O v indica que o parâmetro é um vetor (ou ponteiro para um vetor) **(Figura 7)**.

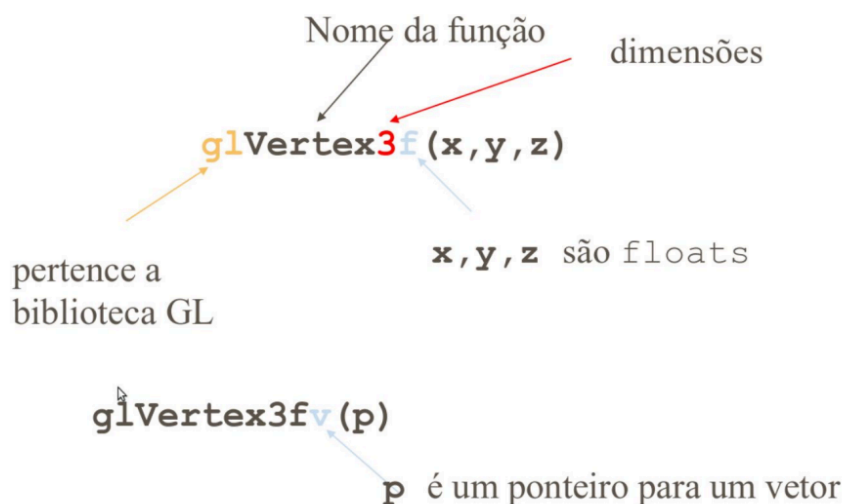


Figura 7 - Código Explicação.

As funções da OpenGL seguem uma estrutura padronizada que facilita seu reconhecimento. Ambas as funções pertencem à biblioteca GL (OpenGL) .

Existem muitas outras formas primitivas como polilinhas, polígonos, etc.

Polilinha (GL_LINE_STRIP, GL_LINE_LOOP): conjunto de segmentos de reta conectados, podendo formar um contorno aberto ou fechado.

• Demais primitivas disponíveis (**Figura 8**):

- GL_POLYGON
- GL_TRIANGLES
- GL_QUADS
- GL_TRIANGLE_STRIP
- GL_TRIANGLE_FAN
- GL_QUAD_STRIP

Uma curiosidade é que GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN e GL_QUAD_STRIP são amplamente utilizados na construção de objetos em 3D, pois facilitam a criação de malhas formadas por polígonos conectados.

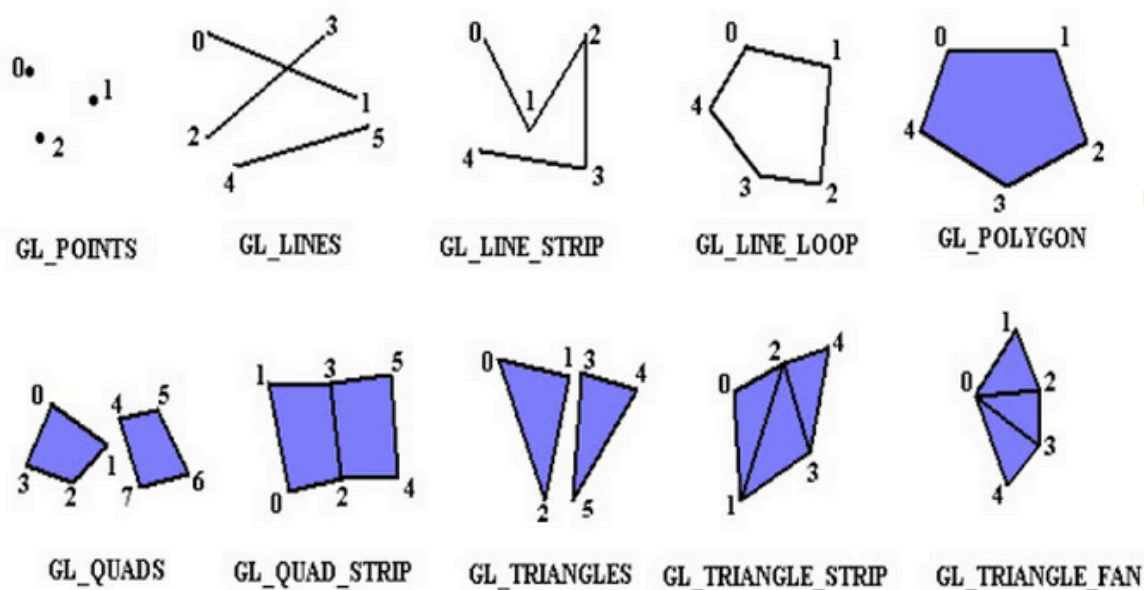


Figura 8 - Várias primitivas geométricas.

3 - Primeiro exemplo: Nosso “Hello World” com OpenGL

Por fim, vamos finalmente ver funcionando tudo que aprendemos nessa aula. Basta seguir o passo a passo:

1. **Crie um novo projeto** no Code::Blocks.
2. **Volte no material e siga o passo a passo do código mostrado.**
3. **Configuração do linker (lembrem-se que SEMPRE ao criar um novo projeto vocês devem configurar o linker ou então não irá funcionar):**
4. **Windows:**
 - a. `-lopengl32 -lglu32 -lfreetype`
5. **Linux:**
 - a. `-lGL -lglut -lGLU`
6. **Compile e execute:** Se tudo estiver correto, Nosso “Hello World” com OpenGL será exibido.

Diferente do exemplo do “Hello World”, existem outras formas mais avançadas que podem ser feitas no decorrer da disciplina. Como por exemplo, a criação de uma esfera formada apenas pelos seus vértices, ou um cubo onde cada face possui a sua própria cor, ambos os objetos realizando uma rotação em torno de seus respectivos eixos. **(Figura 9):**



Figura 9 - Outras possíveis aplicações.